Stack Learning Path & Revision Plan

Goal:

By the end of this session, you should:

- Understand stacks fundamentally
- Efficiently implement stacks in Python
- Solve typical and advanced interview problems related to stacks
- Master the reasoning and optimization skills needed for interview success
- Retain concepts clearly for revision

Day 1: Basics to Intermediate (8 hours)

1. Fundamentals & Theory (1 hr)
2. Stack operations (1 hr)
3. Python Implementation (1 hr)
4. Edge-cases & Error handling (1 hr)
5. Basic Interview Problems (2 hrs)
6. Optimizations & Time Complexity (1 hr)
7. Revision and Note-Making (1 hr)

Day 2: Intermediate to Advanced (8 hours)

1. Stack Variations (1 hr)
2. Advanced Python implementations (1 hr)
3. Advanced Problems (2 hrs)
4. Real-life scenarios (1 hr)
5. System Design and Stack (1 hr)
6. Interview Tips & Tricks (1 hr)
7. Final Revision (1 hr)

Stack Introduction:

Stack (LIFO: Last-In-First-Out)

Operations:

- Push: Add item
- Pop: Remove item
- Peek: Check top item
- isEmpty: Check if empty

Advantages: Fast ops, easy to implement

Disadvantages: Limited access, slow search

Analogy: Plate stacking, Undo feature, Function calls

Stack Implementation in Python:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        raise IndexError("Pop from empty stack!")

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        raise IndexError("Peek from empty stack!")

    def is_empty(self):
        return len(self.stack) == 0
```

Revision Plan:

- End of each day: Create concise notes summarizing daily learnings.
- Include definitions, examples, and complexity analysis.
- Final revision: Review notes, practice mixed-level questions, recall complexity, practice verbal explanations.